

CA-App-Be API Reference

Ready-to-run curl commands for every endpoint

About this API: CA-App-Be is the backend for a CA (Chartered Accountant) directory and lead-management platform - it handles admin authentication, CA submission records (creation, search, CSV import, statistics), a services/sub-services catalog, WhatsApp message templates, and WhatsApp-based conversations and notifications (including bulk email). All 55 routes are listed below, grouped by module, exactly as implemented in the current backend.

How to use this file: set the two shell variables below to your deployed server's URL and a JWT token, then copy/paste any command as-is. Get a token by running the **POST /auth/register** or **POST /auth/login** command first and copying the data . token field from the response into \$TOKEN. Every route is prefixed with /api/v1 - already included in BASE_URL below.

```
# Fill these in, then export them in your shell before running any command below:
export BASE_URL="https://your-deployed-server.example.com/api/v1"
export TOKEN="paste-your-jwt-here-after-calling-login"
```

Lines marked in orange below are known, pre-existing behaviors/bugs in the current backend (verified through direct testing) - not something wrong with your deployment.

Auth & Admin

/auth/*, /adminUsers

POST /auth/register

Register a new admin account. Defaults to the 'admin' role if none is given.

```
curl -X POST "${BASE_URL}/auth/register" \
  -H "Content-Type: application/json" \
  -d '{
    "username": "testadmin",
    "email": "testadmin@example.com",
    "password": "TestPassword123!",
    "phoneNumber": "9876543210"
  }'
```

POST /auth/create

Create a new user account (admin-only). Role can be set via ?role= query param.

```
curl -X POST "${BASE_URL}/auth/create?role=operational" \
  -H "Authorization: Bearer ${TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{
    "username": "opsuser",
    "email": "opsuser@example.com",
    "password": "TestPassword123!",
    "phoneNumber": "9876543211"
  }'
```

POST /auth/login

Log in with a username or email + password. Returns a JWT to use as \$TOKEN below.

```
curl -X POST "${BASE_URL}/auth/login" \
-H "Content-Type: application/json" \
-d '{
  "identifier": "testadmin",
  "password": "TestPassword123!"
}'
```

DELETE /auth

Delete your own account (the account behind \$TOKEN).

```
curl -X DELETE "${BASE_URL}/auth" \
-H "Authorization: Bearer ${TOKEN}"
```

DELETE /auth/delete-user

Delete another user's account (admin-only).

```
curl -X DELETE "${BASE_URL}/auth/delete-user" \
-H "Authorization: Bearer ${TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "targetUserId": "<mongo-object-id>"
}'
```

PUT /auth/update-user

Update another user's account (admin-only).

```
curl -X PUT "${BASE_URL}/auth/update-user" \
-H "Authorization: Bearer ${TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "targetUserId": "<mongo-object-id>",
  "role": "admin"
}'
```

PUT /auth/change-password

Change the current user's password.

NOTE: Known issue: this route is registered but never implemented server-side - it currently returns 404 "Route not found" regardless of input.

```
curl -X PUT "${BASE_URL}/auth/change-password" \
-H "Authorization: Bearer ${TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "oldPassword": "TestPassword123!",
  "newPassword": "NewPassword123!"
}'
```

POST /auth/reset-password-request

Request a password reset OTP by email.

NOTE: Known issue: the reset OTP is generated but never actually saved to the database (a schema gap), so verify/reset will always fail afterward.

```
curl -X POST "${BASE_URL}/auth/reset-password-request" \
-H "Content-Type: application/json" \
-d '{
  "email": "testadmin@example.com"
}'
```

POST /auth/verify-reset-token

Verify a password reset OTP.

```
curl -X POST "${BASE_URL}/auth/verify-reset-token" \
-H "Content-Type: application/json" \
-d '{
  "email": "testadmin@example.com",
  "token": "123456"
}'
```

POST /auth/reset-password

Reset a password using a verified OTP.

```
curl -X POST "${BASE_URL}/auth/reset-password" \
-H "Content-Type: application/json" \
-d '{
  "email": "testadmin@example.com",
  "token": "123456",
  "newPassword": "NewPassword123!"
}'
```

POST /auth/google/callback

Sign in or sign up via Google OAuth.

NOTE: Security note: the server does not verify the Google credential - it trusts whatever email is in the request body. Also, only the FIRST Google sign-in ever succeeds; every subsequent new user fails (duplicate-key bug).

```
curl -X POST "${BASE_URL}/auth/google/callback" \
-H "Content-Type: application/json" \
-d '{
  "profile": {
    "email": "someone@example.com"
  }
}'
```

POST /auth/social/google

Alias for the same Google OAuth flow above.

```
curl -X POST "${BASE_URL}/auth/social/google" \
-H "Content-Type: application/json" \
-d '{
  "profile": {
    "email": "someone@example.com"
  }
}'
```

GET /adminUsers

List all admin users with role counts.

```
curl -X GET "${BASE_URL}/adminUsers?role=admin" \
-H "Authorization: Bearer ${TOKEN}"
```

CA Submissions

/ca/submissions, /ca/search, /ca/stats, /ca/import, /ca/validate-contacts

POST /ca/import

Import CA records from a CSV/Excel file (commits directly).

```
curl -X POST "${BASE_URL}/ca/import" \
-H "Authorization: Bearer ${TOKEN}" \
```

```
-F "file=@/path/to/records.csv"
```

POST /ca/import/preview

Dry-run an import: validates the file, writes nothing.

```
curl -X POST "${BASE_URL}/ca/import/preview" \  
-H "Authorization: Bearer ${TOKEN}" \  
-F "file=@/path/to/records.csv"
```

POST /ca/import/confirm

Commit a previously-previewed import (re-submit the file).

```
curl -X POST "${BASE_URL}/ca/import/confirm" \  
-H "Authorization: Bearer ${TOKEN}" \  
-F "file=@/path/to/records.csv"
```

POST /ca/submissions

Create a CA submission manually.

```
curl -X POST "${BASE_URL}/ca/submissions" \  
-H "Authorization: Bearer ${TOKEN}" \  
-H "Content-Type: application/json" \  
-d '{  
  "name": "Rajesh Kumar",  
  "mobile": "9876543210",  
  "email": "rajesh@example.com",  
  "state": "Delhi",  
  "city": "New Delhi"  
'
```

GET /ca/submissions

List CA submissions with pagination and filters.

```
curl -X GET "${BASE_URL}/ca/submissions?page=1&limit=10&state=Delhi" \  
-H "Authorization: Bearer ${TOKEN}"
```

GET /ca/submissions/

Get a single CA submission by ID.

```
curl -X GET "${BASE_URL}/ca/submissions/<id>" \  
-H "Authorization: Bearer ${TOKEN}"
```

PUT /ca/submissions/

Update a CA submission.

```
curl -X PUT "${BASE_URL}/ca/submissions/<id>" \  
-H "Authorization: Bearer ${TOKEN}" \  
-H "Content-Type: application/json" \  
-d '{  
  "city": "Mumbai"  
'
```

DELETE /ca/submissions/

Delete a CA submission.

```
curl -X DELETE "${BASE_URL}/ca/submissions/<id>" \  
-H "Authorization: Bearer ${TOKEN}"
```

DELETE /ca/submissions/bulk

Bulk delete CA submissions by ID.

```
curl -X DELETE "${BASE_URL}/ca/submissions/bulk" \
-H "Authorization: Bearer ${TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "ids": ["<mongo-object-id-1>", "<mongo-object-id-2>"]
}'
```

GET /ca/search

Full-text search across CA submissions.

```
curl -X GET "${BASE_URL}/ca/search?q=Rajesh&state=Delhi&page=1&limit=10" \
-H "Authorization: Bearer ${TOKEN}"
```

GET /ca/stats

Aggregate statistics: totals by state, city, source, services.

```
curl -X GET "${BASE_URL}/ca/stats" \
-H "Authorization: Bearer ${TOKEN}"
```

GET /ca/stats/services

Per-service counts and percentages.

```
curl -X GET "${BASE_URL}/ca/stats/services" \
-H "Authorization: Bearer ${TOKEN}"
```

GET /ca/master-services

Public list of active services/sub-services.

```
curl -X GET "${BASE_URL}/ca/master-services"
```

PATCH /ca/submissions//active

Toggle or set a submission's active status.

NOTE: Security note: this endpoint has no authentication middleware wired up at all, unlike every sibling route - it works even without \$TOKEN.

```
curl -X PATCH "${BASE_URL}/ca/submissions/<id>/active" \
-H "Authorization: Bearer ${TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "isActive": false
}'
```

GET /ca/by-service/

List CAs offering a specific service.

```
curl -X GET "${BASE_URL}/ca/by-service/<serviceKey>?page=1&limit=10" \
-H "Authorization: Bearer ${TOKEN}"
```

GET /ca/health

Health check for the CA Submissions module.

```
curl -X GET "${BASE_URL}/ca/health"
```

POST /ca/validate-contacts

Validate email or WhatsApp numbers for a set of CA IDs.

```
curl -X POST "${BASE_URL}/ca/validate-contacts" \
-H "Authorization: Bearer ${TOKEN}" \
-H "Content-Type: application/json" \
-d '{
```

```
    "ids": ["<mongo-object-id-1>"],
    "validationType": "email"
  },
```

Services

/ca/services/*

POST /ca/services

Create a new service (optionally with inline sub-services).

```
curl -X POST "${BASE_URL}/ca/services" \
  -H "Authorization: Bearer ${TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Income Tax",
    "subServices": [
      { "name": "Compliance" },
      { "name": "Tax Planning" }
    ]
  }'
```

GET /ca/services

List all services, including inactive/pending ones.

```
curl -X GET "${BASE_URL}/ca/services" \
  -H "Authorization: Bearer ${TOKEN}"
```

GET /ca/services/

Get a single service by ID.

```
curl -X GET "${BASE_URL}/ca/services/<id>" \
  -H "Authorization: Bearer ${TOKEN}"
```

PUT /ca/services/

Update a service's name, alias, or active status.

```
curl -X PUT "${BASE_URL}/ca/services/<id>" \
  -H "Authorization: Bearer ${TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Income Tax Filing"
  }'
```

PATCH /ca/services//active

Toggle or set a service's active status.

```
curl -X PATCH "${BASE_URL}/ca/services/<id>/active" \
  -H "Authorization: Bearer ${TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{
    "isActive": false
  }'
```

POST /ca/services//sub-services

Add a sub-service to an existing service.

```
curl -X POST "${BASE_URL}/ca/services/<id>/sub-services" \
  -H "Authorization: Bearer ${TOKEN}" \
```

```
-H "Content-Type: application/json" \
-d '{
  "name": "GST Filing"
}'
```

PUT /ca/services//sub-services/

Update a sub-service.

```
curl -X PUT "${BASE_URL}/ca/services/<id>/sub-services/<subServiceId>" \
-H "Authorization: Bearer ${TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "name": "GST Filing & Returns"
}'
```

PATCH /ca/services//sub-services//active

Toggle or set a sub-service's active status.

```
curl -X PATCH "${BASE_URL}/ca/services/<id>/sub-services/<subServiceId>/active" \
-H "Authorization: Bearer ${TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "isActive": false
}'
```

WhatsApp Templates

/whatsapp/templates*

POST /whatsapp/templates

Create a WhatsApp template.

NOTE: Known issue: this endpoint always fails (500) - the server reads the raw request body incorrectly and never sees the fields above.

```
curl -X POST "${BASE_URL}/whatsapp/templates" \
-H "Authorization: Bearer ${TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "template_name": "welcome_template",
  "preview": "Hello {{1}}, welcome!",
  "bodyParams": [{ "label": "name", "type": "string" }]
}'
```

GET /whatsapp/templates

List WhatsApp templates.

```
curl -X GET "${BASE_URL}/whatsapp/templates?activeOnly=true" \
-H "Authorization: Bearer ${TOKEN}"
```

GET /whatsapp/templates/

Get a single template by ID.

```
curl -X GET "${BASE_URL}/whatsapp/templates/<id>" \
-H "Authorization: Bearer ${TOKEN}"
```

PUT /whatsapp/templates/

Update a template.

NOTE: Known issue: this endpoint returns 200 but silently changes nothing, for the same reason as POST above.

```
curl -X PUT "${BASE_URL}/whatsapp/templates/<id>" \
-H "Authorization: Bearer ${TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "preview": "Updated preview text"
}'
```

DELETE /whatsapp/templates/

Delete a template.

```
curl -X DELETE "${BASE_URL}/whatsapp/templates/<id>" \
-H "Authorization: Bearer ${TOKEN}"
```

PATCH /whatsapp/templates//toggle

Toggle a template's active status.

NOTE: Note: always flips the current value - it does not accept an explicit {isActive: true/false} the way the CA Submissions/Services toggles do.

```
curl -X PATCH "${BASE_URL}/whatsapp/templates/<id>/toggle" \
-H "Authorization: Bearer ${TOKEN}"
```

Conversations

/conversations* (WhatsApp backend proxy)

GET /conversations

List WhatsApp conversations.

```
curl -X GET "${BASE_URL}/conversations" \
-H "Authorization: Bearer ${TOKEN}"
```

GET /conversations/search

Search conversations by query.

```
curl -X GET "${BASE_URL}/conversations/search?q=9876543210" \
-H "Authorization: Bearer ${TOKEN}"
```

GET /conversations//messages

Get messages for one conversation.

```
curl -X GET "${BASE_URL}/conversations/<userId>/messages?limit=50" \
-H "Authorization: Bearer ${TOKEN}"
```

GET /conversations//history

Get conversation history with a date range.

NOTE: Known issue: always returns 500 - the route points at a controller method that was never implemented.

```
curl -X GET "${BASE_URL}/conversations/<userId>/history" \
-H "Authorization: Bearer ${TOKEN}"
```

Notifications

/messages/*, /notifications/*, /emailLogs

POST /messages/send

Send a single WhatsApp text message.

```
curl -X POST "${BASE_URL}/messages/send" \
-H "Authorization: Bearer ${TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "to": "+919876543210",
  "text": "Hello there",
  "user_name": "Priya"
}'
```

POST /messages/sendTemplate

Send a single WhatsApp template message.

```
curl -X POST "${BASE_URL}/messages/sendTemplate" \
-H "Authorization: Bearer ${TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "to": "+919876543210",
  "name": "welcome_template"
}'
```

POST /messages/sendBulkTemplate

Send a WhatsApp template to multiple recipients.

```
curl -X POST "${BASE_URL}/messages/sendBulkTemplate" \
-H "Authorization: Bearer ${TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "name": "welcome_template",
  "recipients": [
    { "phone": "+919876543210", "user_name": "Priya" },
    { "phone": "+919876543211" }
  ]
}'
```

POST /notifications/whatsapp/bulk

Send a plain-text WhatsApp message to multiple recipients.

```
curl -X POST "${BASE_URL}/notifications/whatsapp/bulk" \
-H "Authorization: Bearer ${TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "text": "Reminder: your appointment is tomorrow",
  "recipients": ["+919876543210", "+919876543211"]
}'
```

POST /notifications/email/bulk

Send a bulk email to multiple recipients.

```
curl -X POST "${BASE_URL}/notifications/email/bulk" \
-H "Authorization: Bearer ${TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "emails": ["a@example.com", "b@example.com"],
  "subject": "Update",
  "message": "Body text here"
}'
```

GET /emailLogs

List sent-email logs.

```
curl -X GET "${BASE_URL}/emailLogs?status=sent" \  
  -H "Authorization: Bearer ${TOKEN}"
```

Misc

GET /health-check

Generic router-level health check.

```
curl -X GET "${BASE_URL}/health-check"
```